

Building a Robust Part-of-Speech Tagger Using Hidden Markov Models: An Implementation and Error Analysis

Kuong Sivhoung-e20221198

Min Sivee-e20220283

Lun Rathana-e20220368

Applied Mathematics and Statistics
Institute of Technology of Cambodia
Phnom Penh, Cambodia

Abstract

Resolving syntactic ambiguity remains a foundational challenge in natural language processing (NLP). This report details the implementation of a part-of-speech (POS) tagger built from scratch using a hidden Markov model (HMM). Trained and evaluated on the Brown Corpus with a universal tagset, the model achieves a word-level tagging accuracy of 92.47%. To bridge the gap between theoretical modeling and practical application, programmatic solutions such as Laplace (Add- α) smoothing and log-space mathematical transformations were implemented to prevent out-of-vocabulary (OOV) failures and computational underflow. Finally, a rigorous error analysis exposes the theoretical limitations of the first-order Markov assumption, specifically its vulnerability to cascading errors and its inability to model long-distance syntactic dependencies.

1 Introduction

A core challenge in natural language processing (NLP) is the resolution of syntactic ambiguity. Human language is inherently contextual; consequently, many words in the English vocabulary can assume markedly different grammatical roles depending on their surrounding syntax. For instance, the lexical token “watch” functions as a noun in “my watch is broken,” yet acts as an imperative verb in “watch the dog.” Accurately assigning the correct part-of-speech (POS) tag to every word in a sequence is not merely a theoretical exercise; it is an essential preprocessing step for downstream tasks such as syntactic parsing, named-entity recognition, and semantic analysis (Jurafsky and Martin, 2023).

To computationally address this ambiguity, this project implements a hidden Markov model (HMM) (Rabiner, 1989). HMMs are probabilistic sequence models that are highly effective for

tagging because they simultaneously evaluate two dimensions of probability: the likelihood of a word being generated by a grammatical tag (emission), and the likelihood of a tag following the preceding tag (transition). This report details the theoretical formulation, programmatic implementation, and empirical evaluation of a POS tagger, with particular focus on the engineering hurdles of zero-probability emissions and floating-point collapse.

2 Theoretical Background

The hidden Markov model is a statistical generative model in which the system being modeled is assumed to be a Markov process with unobserved (hidden) states. In the context of POS tagging, the observations O are the sequence of visible words in a sentence, and the hidden states S are the underlying POS tags that generate those words.

The objective of the tagger is to find the most probable sequence of hidden states given the observed words. Applying Bayes’ theorem together with the model’s independence assumptions, this reduces to maximizing the joint product of emission and transition probabilities, as expressed in Equation 1.

$$T^* = \arg \max \prod P(w_i | t_i) P(t_i | t_{i-1}) \quad (1)$$

This formulation relies on two simplifying assumptions. First, the first-order Markov assumption states that the probability of a state depends only on the immediately preceding state, $P(t_i | t_{i-1})$, and carries no broader historical context. Second, the output-independence assumption states that the probability of an observed word depends strictly on its current state tag, $P(w_i | t_i)$, independent of surrounding words. These assumptions make the model tractable but, as

Section 5 shows, also bound its accuracy.

3 Methodology

The implementation of the tagger was partitioned into data acquisition, maximum-likelihood parameter estimation, and robust decoding via the Viterbi algorithm.

3.1 Data Acquisition and Partitioning

The model was trained and evaluated using the Brown Corpus (Francis and Kučera, 1979), a standard NLP benchmark, accessed through the Python Natural Language Toolkit (Bird et al., 2009). The universal tagset was applied to collapse the state space into twelve generalized grammatical categories. To enable a robust evaluation of the model’s capacity to generalize to unseen data, the corpus was strictly partitioned: 80% of the sentences were allocated to the training set, while the remaining 20% were held out for testing.

Figure 1 shows the tag distribution. It is heavily skewed NOUN alone accounts for 23.7% of all tokens, and the four most frequent tags cover roughly two thirds of the corpus. The imbalance is significant because it shows that even a simple classifier that assigns each word its most frequently occurring tag can achieve relatively high accuracy. As a result, it provides a strong baseline that any effective model should be able to perform better.

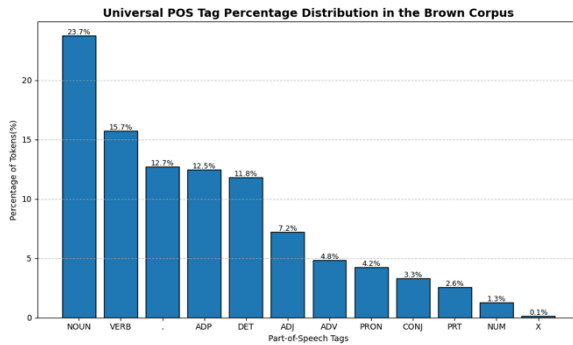


Figure.1. Distribution of the twelve universal POS tags across the Brown Corpus, as a percentage of all tokens.

3.2 Parameter Estimation via MLE

The training phase computes three core probability structures from empirical frequency counts using maximum-likelihood estimation

(MLE): the initial probabilities π , the likelihood of a sentence beginning with a given tag; the transition matrix A , defined in Equation 2; and the emission matrix B , defined in Equation 3. Here $Count(\cdot)$ denotes a corpus frequency count.

$$P(t_j|t_i) = \frac{Count(t_i, t_j)}{Count(t_i)} \quad (2)$$

$$P(w_i|t_i) = \frac{Count(t_i, w_i)}{Count(t_i)} \quad (3)$$

Raw MLE, however, fails when exposed to unobserved data, since any event absent from the training counts receives a true zero probability. This necessitates mathematical smoothing.

3.3 Laplace Smoothing for OOV Words

When evaluating the test set, the model inevitably encounters out-of-vocabulary (OOV) words absent from the training phase. Standard MLE assigns a zero probability to these emissions. Because the sequence probability is a product of individual probabilities, a single zero collapses the entire sequence probability.

To solve this, Laplace (Add- α) smoothing was integrated into emission estimation, as shown in Equation 4. A pseudo-count α is added to every potential emission, and the denominator is expanded by $\alpha|V|$, where $|V|$ is the unique vocabulary size. This reserves probability mass for unseen tokens, allowing the decoder to assign a non-zero fallback score to novel words and continue processing without catastrophic failure.

$$P(w_j|t_i) = \frac{Count(t_i, w_i) + \alpha}{Count(t_i) + \alpha|V|} \quad (4)$$

3.4 Log-Space Transformation and Safety Buffers

The Viterbi algorithm (Viterbi, 1967) decodes the optimal tag sequence by multiplying sequential probabilities. For sentences of typical length, multiplying dozens of fractional probabilities yields infinitesimally small numbers, inducing computational underflow a hardware limitation in which standard floating-point architectures round these values down to an absolute 0.0.

To bypass this constraint, the algorithm was

projected into log-space. Because $\log(xy) = \log x + \log y$, the recursion was rewritten to perform numerically stable addition rather than unstable multiplication, as in Equation 5, where $\delta_t(j)$ is the highest log-probability of any path ending in state j at position t .

$$\delta_t(j) = \max_i [\delta_{t-1}(i) + \log a_{ij}] + \log b_j(o_t) \quad (5)$$

Furthermore, to prevent crashes when evaluating $\log(0)$ —which raises an undefined `ValueError`—an epsilon safety buffer ($\epsilon = 10^{-12}$) was injected into the log wrapper, guaranteeing stable execution across all edge cases.

4 Results

Table 1: Dataset partition and model performance.

Parameter	Value
Corpus	Brown (NLTK)
Tagset	Universal (12)
Total sentences	57,340
Training (80%)	45,872
Testing (20%)	11,468
Decoder	Log Viterbi
Accuracy	92.47%

The hidden Markov model, smoothed with an Add- α pseudo-count and processed via a log-space Viterbi decoder, achieved a final word-level tagging accuracy of 92.47% on the held-out test set (Table 1). This high baseline indicates that the mathematical framework of combining transition and emission probabilities successfully maps the majority of standard English grammatical structures.

5 Error Analysis

While the tagger performs well on localized, highly predictable grammatical structures, an examination of the 7.53% error rate exposes the theoretical constraints of the first-order Markov assumption. Because the model restricts its context window to a single preceding state ($k = 1$), it suffers from a form of syntactic shortsightedness.

5.1 The Cascading Error Phenomenon

A fundamental flaw of the Markov assumption

is that a single incorrect prediction heavily biases the subsequent prediction, producing a chain reaction of failures.

Sentence: “What’s below the water-line interests me also.”

True tags: ... [DET] [NOUN] [VERB] ...

Predicted: ... [DET] [ADJ] [NOUN] ...

When the decoder reached “water-line,” it incorrectly emitted an adjective (ADJ) tag owing to sparse training data. Because the system carries only first-order memory, its prediction for “interests” was predicated entirely on the false assumption that the current state was ADJ. Since the transition from an adjective to a noun is overwhelmingly probable, the transition matrix forced the model to tag “interests” as a NOUN rather than a VERB. The model lacked the global context to identify “interests” as the main verb of the independent clause.

5.2 Over-Reliance on Dominant Bigrams

The HMM frequently overrides valid lexical emissions in favor of statistically dominant structural bigrams.

Sentence: “The Beech Pasture had suddenly become valuable.”

True tags: [DET] [NOUN] [NOUN] ...

Predicted: [DET] [ADJ] [NOUN] ...

Here, “Beech” functions as a noun adjunct. Recognizing the surrounding DET ... NOUN structure, however, the model defaulted to the DET → ADJ → NOUN sequence. Because constructions such as “the green pasture” are far more frequent in the corpus than double-noun sequences, the weight of the bigram transition overpowered the emission probability of “Beech,” forcing a misclassification.

5.3 Failures in Parallel Structure

Sentence: “Red was small and fine-boned...”

True tags: [NOUN] [VERB] [ADJ] [CONJ] [ADJ]

Predicted: [PRON] [VERB] [ADJ] [CONJ] [NOUN]

The model failed on “fine-boned” because the conjunction “and” acts as a transition pivot. A higher-order parser would recognize the parallel structure [ADJ] and [ADJ] (“small and fine-boned”). Looking only one step backward at CONJ, however, the HMM lacks the persistent memory required to reference the adjective “small”

preceding the conjunction. Furthermore, “Red” was misclassified as a pronoun because the initial-probability distribution π heavily biases sentence-initial capitalized words toward the pronoun class.

6 Conclusion

This project successfully bridged theoretical probabilistic models and functional software engineering to implement an HMM part-of-speech tagger from scratch. By integrating Laplace smoothing and log-space mathematical transformations, the model overcame the standard programmatic pitfalls of OOV emissions and floating-point underflow.

Achieving 92.47% accuracy demonstrates that MLE-derived HMMs serve as robust baselines for sequence tagging. The qualitative error analysis, however, explicitly defines the mathematical ceiling of this methodology. The first-order Markov assumption critically restricts the model’s contextual awareness, blinding it to long-distance grammatical agreements and leaving it susceptible to cascading failures.

References

- D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft. Stanford, CA: Stanford University, 2023. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>
- L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257-286, Feb. 1989.
- W. N. Francis and H. Kučera, *Computational Analysis of Present-Day American English*. Providence, RI: Brown University Press, 1967.
- A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260-269, April 1967.
- S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. Beijing: O'Reilly Media, 2009.